MyCodeSchool

Spring Data JPA — adnotările folosite

One Model · fiecare adnotare din proiect: unde stă și ce face

Unde stă o adnotare · deasupra lucrului pe care îl descrie

```
@Entity                      ←— pe clasă
@Table(name = "products")
public class Product {

    @Id                      ←— pe câmp
    private UUID id;
}
```

```
public interface ProductRepository ... {

    @Query("select ...")     ←— pe metodă
    List<Product> metoda(
        @Param("x") int x);  ←— pe parametru
}
```

Din Product iese tabela products · citești de la stânga la dreapta

```
@Id @GeneratedValue(strategy = UUID)
UUID id;
@Column(nullable = false, length = 100)
String name;
@Column(nullable = false, unique = true, length = 8)
String sku;
@Enumerated(EnumType.STRING)
Category category;
@Column(nullable = false, precision = 10, scale = 2)
BigDecimal price;
@Column(name = "added_on", nullable = false)
LocalDate addedOn;
```

```
──► id        binary(16)     PRIMARY KEY

──► name      varchar(100)   NOT NULL

──► sku       varchar(8)     NOT NULL  UNIQUE

──► category  enum('COFFEE','EQUIPMENT',...)

──► price     decimal(10,2)  NOT NULL

──► added_on  date           NOT NULL
```

Modelul devine tabelă · jakarta.persistence · pe Product, le primești gata

`@Entity`	clasă	clasa e salvată în bază: un obiect = un rând
`@Table(name = "products")`	clasă	numele tabelei; fără ea, tabela ia numele clasei
`@Id`	câmp	cheia primară
`@GeneratedValue(strategy = GenerationType.UUID)`	câmp	id-ul îl generează Hibernate la `save`, nu îl pui tu
`@Column(...)`	câmp	regulile coloanei: `name` (alt nume), `nullable = false`, `unique = true`, `length`, `precision` + `scale`
`@Enumerated(EnumType.STRING)`	câmp	enum-ul se salvează ca text (`COFFEE`), nu ca număr

Validări · jakarta.validation.constraints · verificate în Java, înainte de insert (ex. 3)

`@NotBlank`	`name`, `sku`	text nenul, cu cel puțin un caracter vizibil; `"  "` e respins
`@NotNull`	`category`, `price`, `addedOn`	valoarea nu poate lipsi
`@Size(max = 100)`	`name`	lungimea textului
`@Pattern(regexp = "[A-Z]{3}-\\d{4}")`	`sku`	textul are forma dată: 3 litere mari, cratimă, 4 cifre; `abc` e respins
`@DecimalMin("0.01")`	`price`	cel mai mic preț acceptat; `0.00` e respins
`@Digits(integer = 8, fraction = 2)`	`price`	cel mult 8 cifre înainte de virgulă și 2 după
`@Min(0)`	`stock`	cel mai mic număr acceptat; −1 e respins
`@PastOrPresent`	`addedOn`	data nu poate fi în viitor

```
message = "Name is required"   textul pe care îl afișezi din ConstraintViolationException

@NotNull                 regulă în Java  ──► oprește produsul înainte de bază, cu mesajul tău
@Column(nullable=false)  regulă în MySQL ──► ultima plasă, dacă cineva scrie în tabelă pe lângă aplicație
```

Lombok · lombok · scrie cod la compilare

`@Getter @Setter`	clasă	`getName()`, `setPrice(...)` pentru fiecare câmp
`@NoArgsConstructor`	clasă	constructorul fără parametri, cerut de Hibernate când citește din bază
`@ToString`	clasă	`Product(id=..., name=..., ...)` la afișare (ex. 2)

Spring · org.springframework

`@SpringBootApplication`	`OneModelApplication`	pornește Spring și caută clasele adnotate din pachetul `app`
`@Component`	`ViewProduct`	Spring creează obiectul și îi dă `ProductRepository` prin constructor

În repository · le scrii tu, de la ex. 7

`@Query("...")`	metodă	interogarea scrisă de tine, în JPQL; cu `nativeQuery = true`, în SQL (ex. 7)
`@Param("maxPrice")`	parametru	leagă parametrul de `:maxPrice` din interogare (ex. 7)
`@Modifying(clearAutomatically = true)`	metodă	interogarea e un `update` / `delete`, nu un `select` (ex. 8)
`@Transactional`	metodă	rulează într-o tranzacție; cerută la `@Modifying` și la `deleteBy...` (ex. 8, 9) · importul din `org.springframework.transaction.annotation`